

CODE AUDIT & QUALITY REPORT

Fawnix Verse

ERP Platform — Architecture, Standards, Redundancy & Ownership

143.6k	~12%	13	0
lines analyzed	duplicated	verified bugs	tests

Scope 1,189 source files · frontend + 15 microservices + ML +
infra

Commit 5939027 · Date 2026-07-14

Method git + static analyzers, hotspot deep-dive, bugs verified in
source

Contents

1. Executive Summary	7. Frontend & Backend Findings
2. System Architecture	8. Infrastructure & Security Risk Register
3. Codebase Metrics	9. Coding Standards (condensed)
4. Ownership & Attribution	10. How To Fix It Properly
5. Verified Bugs	11. Remediation Roadmap
6. Redundancy & Copy-Paste	12. Appendix — Method & Data

Severity legend: P0 critical/security P1 high P2 medium P3 low/tech-debt

1 • Executive Summary

Fawnix Verse is a capable, feature-rich ERP platform with a sound high-level architecture — feature-per-service backend, per-service databases, a modern typed frontend. The risks are not architectural. They are in **security hygiene, an incomplete product merge, the total absence of tests, and CI/CD that deploys unverified code straight to production.**

5

P0 CRITICAL

16,230

DUPLICATED LOC

232

COMMITTS · 2 AUTHORS 98%

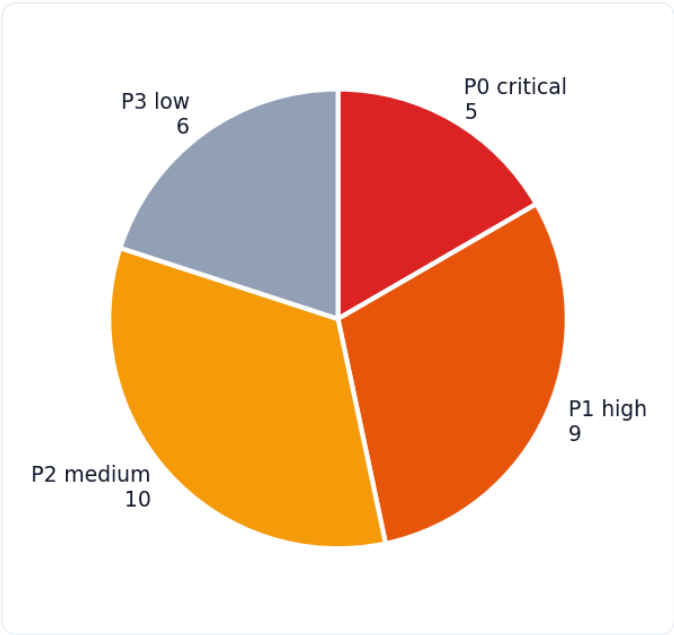
97.8k

AUTHORED LOC

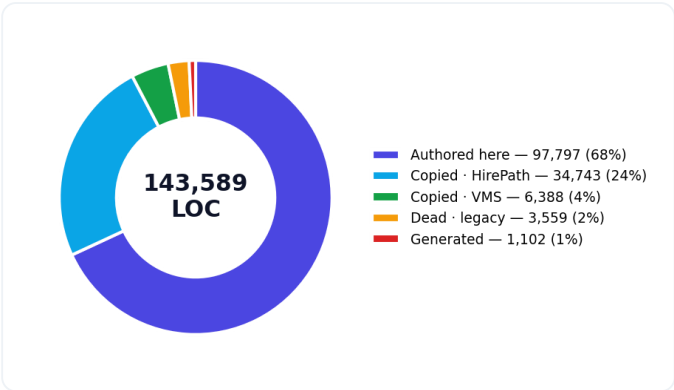
Act first. Live Google/Gemini API keys are committed to git, and several services ship usable default credentials (JWT_SECRET, Admin@123, minioadmin). Rotate and remove before anything else.

The three themes

- Incomplete HirePath → Fawnix merge.** 8 backend services still under `com.hirepath`, missing exception handlers, residual artifacts; the "visitor management" frontend is a foreign body (own auth, own HTTP client, hardcoded ngrok URL, excluded from type-checking).
- Maintainability drag.** ~12% duplicated code — a security stack copy-pasted into 15 services, audit fields into 46 entities, DTOs across services — plus 1,000-4,000-line god-components.
- No safety net.** Zero tests, no CI gates, root containers, no resource limits, no rollback.



Headline findings by severity

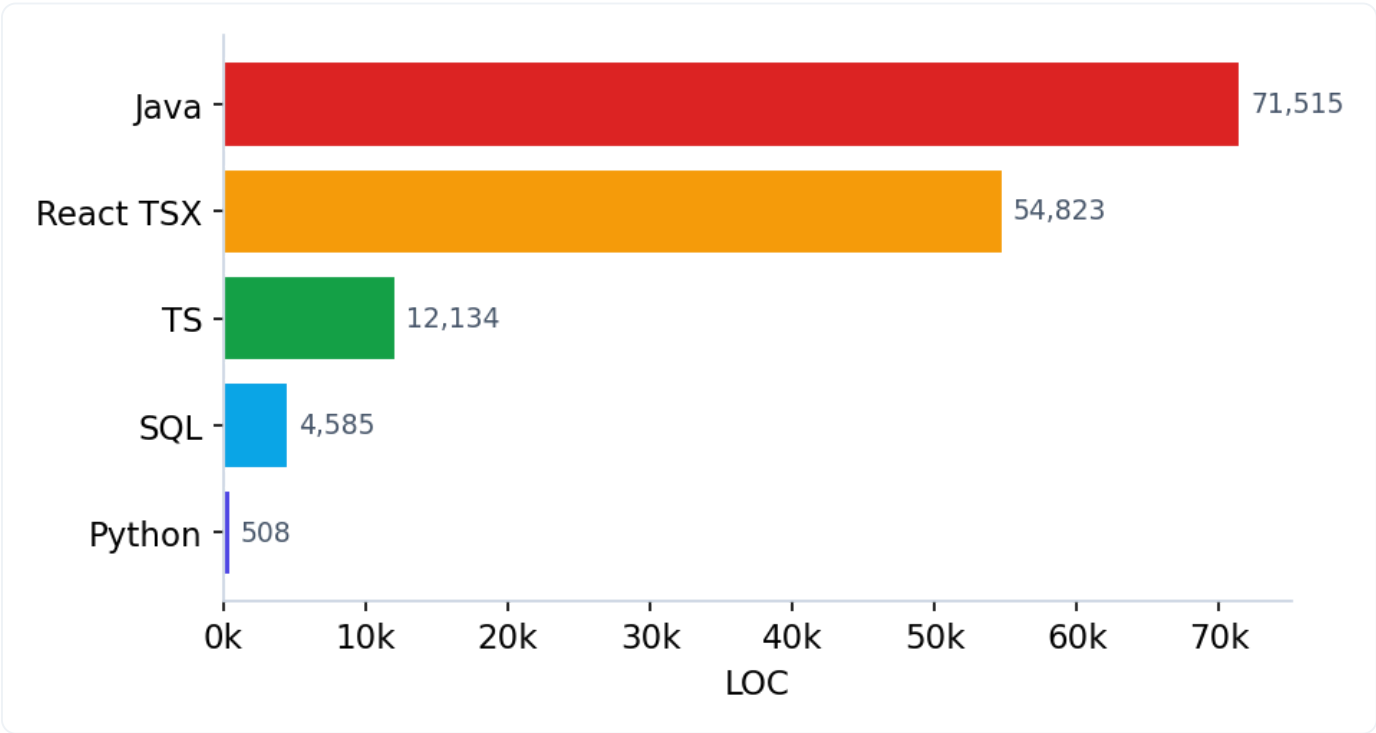


Where the code came from

2 · System Architecture

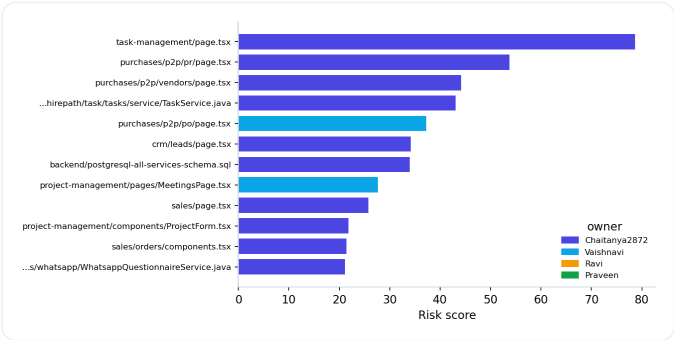
React 19 SPA → Spring Cloud Gateway (JWT, CORS) → Eureka discovery → 15 Spring Boot services → PostgreSQL (one DB per service), Redis, MinIO, and a Python speech-to-text service. Deployed as a 24-container Docker Compose stack on a single EC2 host.

LAYER	STACK	NOTES
Frontend	React 19, TS strict, Vite 7, Tailwind v4, shadcn/ui, TanStack Query, RHF+Zod	17 feature modules
Gateway/Discovery	Spring Cloud Gateway, Eureka, OpenFeign	JWT validated at gateway <i>and</i> per service
Services	Spring Boot 3.3.5, Java 17, JPA + Flyway, JWT	identity, crm, sales, inventory, procurement, task, project + 8 HirePath
Data	PostgreSQL 16, Redis 7, MinIO	DB-per-service; shared superuser (risk)
ML	Python speech-to-text (faster-whisper)	CRM call-recording transcription
Infra	Docker Compose x24, nginx, Caddy (TLS), GitHub Actions	single-host; push-to-master deploy

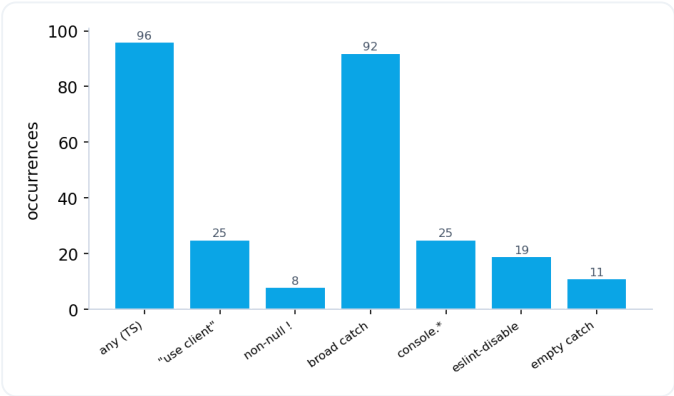


Lines of code by language

3 - Codebase Metrics



Top hotspots by risk score (size × complexity × issues × churn)



Code smells across the repo

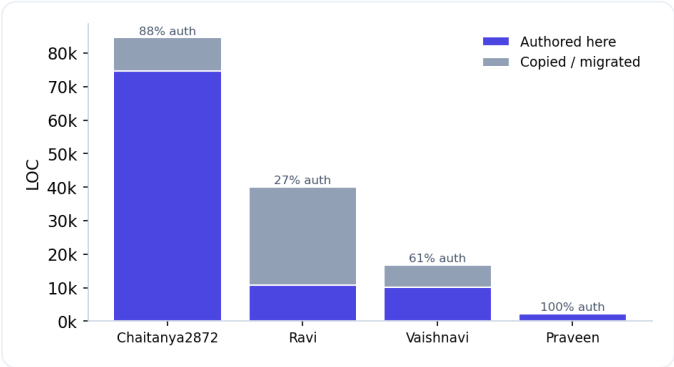
Highest-risk files

RISK	FILE	LOC	CX	ISS	OWNER
78.8	src/modules/task-management/page.tsx	4,129	339	6	Chaitanya2872
53.8	src/modules/purchases/p2p/pr/page.tsx	2,973	266	2	Chaitanya2872
44.3	src/modules/purchases/p2p/vendors/page.tsx	2,240	218	4	Chaitanya2872
43.2	task-service/.../hirepath/task/tasks/service/TaskService.java	2,191	278	1	Chaitanya2872
37.3	src/modules/purchases/p2p/po/page.tsx	2,262	132	0	Vaishnavi Nerella
34.3	src/modules/crm/leads/page.tsx	1,489	114	1	Chaitanya2872
34.1	backend/postgresql-all-services-schema.sql	1,497	265	0	Chaitanya2872
27.7	src/modules/project-management/pages/MeetingsPage.tsx	1,878	95	0	Vaishnavi Nerella
25.9	src/modules/sales/page.tsx	1,845	72	3	Chaitanya2872
21.9	src/modules/project-management/components/ProjectForm.tsx	1,181	84	0	Chaitanya2872
21.5	src/modules/sales/orders/components.tsx	1,737	52	1	Chaitanya2872
21.2	crm-service/.../fawnix/crm/integrations/whatsapp/WhatsappQuestionnaireService.java	824	135	5	Chaitanya2872
20.7	src/modules/sales/orders/page.tsx	831	44	4	Chaitanya2872
19.7	src/modules/users/page.tsx	1,238	50	0	Vaishnavi Nerella
19.7	src/modules/visitor management/services/visitorRequestService.ts	394	112	12	Vaishnavi Nerella

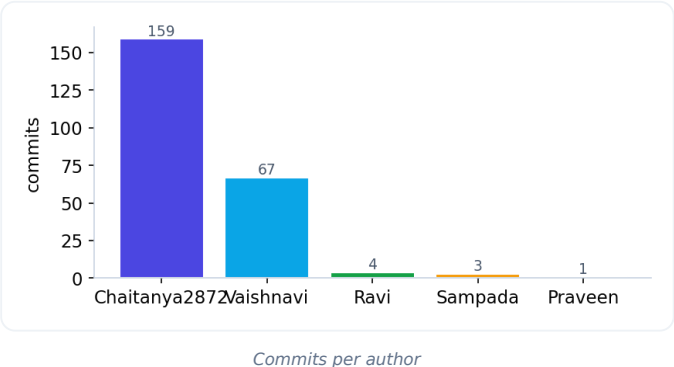
Cx = cyclomatic-ish decision count. The top hotspots are large god-components, not the buggyest files — size and complexity dominate their risk. Recruitment files rank on issue-density instead.

4 · Ownership & Attribution

Read carefully. "Owner" = primary line-author via `git log --numstat` — *not* proof of who wrote the logic. 98% of commits come from two people, and much attributed code is **copied, not authored**. This routes fixes to the right area; it is not a blame ranking.



LOC per author — authored vs copied/migrated



Commits per author

AUTHOR	FILES	LOC	AUTHORED	COPIED/MIGRATED	OWNS
Chaitanya2872	653	84,591	88%	6% hirepath + 4% dead	Fawnix services, main frontend, god-components, most bugs
Ravi-Shankar-ACS migrated	450	40,108	27%	73% HirePath bulk-migration	recruitment FE, cross-service DTO copies (highest defect density)
Vaishnavi Nerella	78	16,705	61%	38% copied VMS	P2P PO page (tax bug), Meetings, Users
Praveen	8	2,185	100%	—	project-management page

Migration distortion. Ravi's footprint is one 394-file / 31,916-line commit ("integrate HRMS modules"). Defects in that migrated code may pre-date this repo — the owner is accountable for what ships, but did not necessarily author it. Such findings are tagged migrated.

5 · Verified Bugs

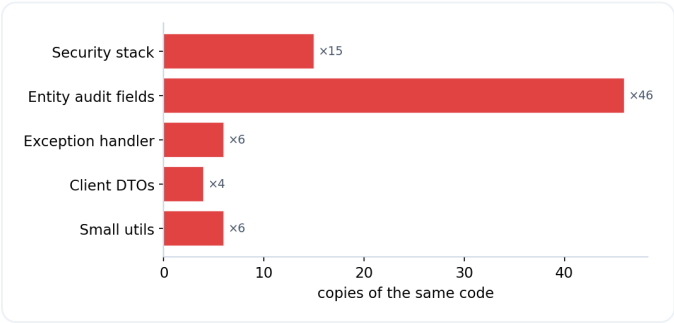
Each was read and confirmed in source. Ranked by blast radius. Security-class issues are in §8.

SEV	BUG	LOCATION	OWNER
P1	IOTIQ purchase orders double-taxed (CGST+SGST unconditional + IGST)	po/page.tsx:258	Vaishnavi Nerella
P1	@Transactional no-op on protected+self-invoked WhatsApp handler	WhatsappQuestionnaireService.java:231	Chaitanya2872
P1	External HTTP (inventory reserve/validate) inside DB transaction	SalesOrderService.java:506,646	Chaitanya2872
P1	Non-atomic ID gen → duplicate task codes / order numbers	TaskService.java:2072	Chaitanya2872
P1	Batch lead import has no transaction → partial commits	LeadImportService.java:69	Chaitanya2872
P1	Inventory 'no response' treated as 'in stock'	SalesOrderService.java:506	Chaitanya2872
P1	Salary renders '?12.5L' not '₹12.5L' (encoding)	OffersPage.tsx:23	Ravi-Shankar [migrated]
P1	DnD form builder broken by nested component definition	ApplicationFormBuilderPage.tsx:335	Ravi-Shankar [migrated]
P1	TalentPool page renders 5 hardcoded fake candidates, no API	TalentPoolPage.tsx:13	Ravi-Shankar [migrated]
P2	Payment rejection reason hardcoded	payment/page.tsx:524	Chaitanya2872
P2	Wrong @Transactional import (jakarta not Spring)	TaskService.java:70	Chaitanya2872
P2	parsePermissions 500s on unknown enum value	TaskService.java:1891	Chaitanya2872
P2	localStorage effect clobbers saved draft before data loads	p2p/pr/page.tsx:1242	Chaitanya2872

Flagship: IOTIQ purchase orders are **double-taxed** — CGST+SGST are added unconditionally on top of IGST (po/page.tsx:258), overstating vendor totals by ~18%.

6 • Redundancy & Copy-Paste

~12% of the code (16,230 LOC) is duplicated — the dominant maintainability problem and a direct symptom of copy-paste instead of shared abstractions.



Same code, copied N times

CLUSTER	SCALE	PROPER FIX
Security stack (JwtFilter, JwtService, SecurityConfig...)	15 services / 75 files	common-security module
Entity audit fields (created/updated + accessors)	46 entities	@MappedSuperclass + JPA Auditing
GlobalExceptionHandler + ApiErrorResponse	6 services	common-web module
Cross-service client DTOs (SendNotification, FormSubmission...)	2-4 copies each (89-90%)	versioned *-api-contract module
SalesOrderEntity ≈ QuoteEntity	49% dup	@MappedSuperclass SalesDocumentBase
P2P page helpers/styles/components	4-7 pages	shared p2p components + lib/format.ts
formatDate / trimToNull / normalizeRole	6-14 files	one shared util
backend/src legacy monolith	54 files (dead)	delete

The fix removes ~5,000+ LOC. Create `common-jpa` (@MappedSuperclass + JPA Auditing), `common-web` (exception handler), `common-security` (JWT stack), and a versioned `*-api-contract` module per provider (DTO + Feign interface defined once).

7 · Frontend & Backend Findings

Frontend (structural / standards)

- **God-components** — task-management 4,129 lines / 28 `useState` / 4 file-level `eslint-disable` masking real effect-loop bugs. Decompose by view; use RHF+Zod for forms.
- **No data layer** in migrated modules — inline `useQuery` keys collide; no `hooks.ts/key-factory`; no global `staleTime`. migrated
- ~87 any, 25 no-op "use client", `localStorage` used as a business database (no versioning/TTL/conflict handling).

Backend (performance / structure)

- **HTTP calls inside** `@Transactional` (inventory reserve, WhatsApp send) → split-brain state & pool exhaustion. Use `@TransactionalEventListener(AFTER_COMMIT)`.
- `findAll().stream().filter()` across every read path (TaskService visibility = 2 queries × N tasks). Push predicates to SQL, paginate, preload sets.
- TaskService **god-class** (2,190 lines, 12 deps, 6 domains); file bytes stored as DB BLOB; money/timezone handling; broad/empty catches.

8 · Infrastructure & Security Risk Register

SEV	AREA	FINDING	WHERE
P0	Security	Live Gemini API keys committed to git	ml/.../.env.example
P0	Security	Usable default secrets (JWT, Admin@123, minioadmin, postgres)	*/application.yml
P0	Security	VMS: hardcoded ngrok URL + admin/secret + offline auth-bypass	visitor management/*
P0	Infra	All containers run as root	backend & ml Dockerfiles
P1	Quality	Zero automated tests (front & back)	—
P1	CI/CD	master auto-deploys to prod, no test/lint gate, StrictHostKeyChecking=no	deploy-ec2.yml
P1	Backend	8 HirePath services lack GlobalExceptionHandler	org/recruitment/forms...
P1	Backend	sales→inventory hardcoded localhost:8083 (bypasses Eureka)	SalesOrderService
P1	Backend	hrms-service denyAll() but routed → 403	hrms-service
P1	Infra	PRD serves a Vite dev server, not the built image	PRD/compose.yml
P2	Backend	Incomplete merge: 8 services under com.hirepath + groupId mismatch	backend/services/*
P2	Backend	Security stack copy-pasted ×14; 3 internal-auth schemes	*/security
P2	Frontend	God-components (task page 4,129 lines)	task-management/page.tsx
P2	Infra	No resource limits; shared PG superuser; MinIO root keys as creds	compose.yml

CI/CD. A push to master SSHes into EC2 and runs `docker compose up --build` — no test/lint/scan gate, images built on the prod host, `StrictHostKeyChecking=no`, no staging, no rollback.

9 - Coding Standards (condensed)

AREA	DO	DON'T
React/TS	Function components; typed API; discriminated unions; RHF+Zod; lazy routes	any; "use client"; 400+ line components; localStorage as DB
TanStack Query	Key factory per module; global defaults; invalidate on mutate	Inline string keys; onSuccess on useQuery
Spring	Package-by-feature; DTOs (MapStruct); ProblemDetail; @ConfigurationProperties	Return entities; hardcoded URLs; HTTP in txn; default secrets
Shared code	common-* modules; @MappedSuperclass; versioned contracts	Copy-paste security/DTOs/entities/utils
Docker/CI	Non-root; pinned digests; healthchecks; resource limits; test/lint gate	Root; :latest; build on prod host; secrets in env/`.env.example`
Git	Conventional commits + scope; PR + review; branch protection	Direct-to-master; committed secrets/clutter

Full, cited standards live in [docs/coding-standards/](#).

10 · How To Fix It Properly

The recurring anti-patterns, each with the correct approach (full code in docs/audit/granular/fixing-properly.md):

ANTI-PATTERN (WHAT JUNIORS DID)	PROPER APPROACH
Audit fields copied into 46 entities	@MappedSuperclass BaseAuditEntity + @EnableJpaAuditing
Security/exception code copied into every service	Shared common-security / common-web Maven modules
DTOs copied into each Feign caller	Versioned *-api-contract (DTO + client interface, once)
External HTTP inside a transaction	Commit first, react in @TransactionalEventListener(AFTER_COMMIT)
count()+1 / random IDs	DB sequence (@GeneratedValue(SEQUENCE))
findAll().filter() in Java	Predicate + Pageable in SQL; preload sets once
Inline query keys + any responses	queryKeys.ts + hooks.ts + generated types
18-26 useState per form	react-hook-form + Zod schema

Root cause is the absence of guardrails, not any one developer. Add CI gates (warn→enforce), a first test suite, one reference module to copy, and a duplication check — then the abstractions above stay removed.

11 · Remediation Roadmap

PHASE	WORK	EFFORT	BUYS
1 · Contain	Rotate & purge committed keys; remove default creds; delete ngrok/demo auth	days	P0 security
2 · Guardrails	Add CI gates (tsc, eslint --max-warnings 0, prettier, spotless) warn→enforce; first tests	1 wk	stops regression
3 · Correctness	Fix 13 verified bugs (IOTIQ tax first); add missing exception handlers; fix hardcoded URL	1-2 wk	data integrity
4 · Harden infra	Non-root containers; prod frontend image; resource limits; per-service DB/MinIO creds	1-2 wk	reliability/security
5 · De-duplicate	common-jpa / common-web / common-security + *-api-contract modules (–5k LOC)	2-3 wk	maintainability
6 · Refactor	Split god-components; finish com.hirepath→com.fawnix; delete backend/src + schema file	ongoing	velocity

12 · Appendix — Method & Data

No linters were installed in the environment, so objective signals were produced with git + custom Python analyzers (offline, reproducible):

- **Per-file metrics** — size, cyclomatic-ish decision count, language-specific smells, churn, primary owner, risk score. → `docs/audit/granular/data/metrics.csv` (1,189 rows)
- **Duplication** — a 6-line clone detector across TS/Java (shadcn excluded). → `docs/audit/granular/data/duplication.csv` (1,098 rows)
- **Hotspot deep-dive** — top files by risk + duplication read line-by-line; flagship bugs verified in source.

Companion markdown: `docs/architecture/`, `docs/coding-standards/`, `docs/audit/` (incl. `granular/`). Every number in this report traces to the CSVs above.